

Análisis sobre el Dataset de Indicadores de Ocupabilidad del Ministerio de Comercio Exterior y Turismo

Estudiantes:

Sergio Sebastian Pezo Jimenez
Universidad Nacional de Ingeniería,
Facultad de Ciencias
e-mail: sergio.pezo.j@uni.pe

Arbués Enrique Perez Villegas
Universidad Nacional de Ingeniería,
Facultad de Ciencias
e-mail: arbues.perez.v@uni.pe

Curso:

CC531 Análisis en Macrodatos
Laboratorio 01

Resumen

Se trabajó con los indicadores mensuales de ocupabilidad hotelera de 2019 a junio de 2025. Primero se unificaron siete archivos CSV y se obtuvo una entrada canónica de 38 730 registros. Luego se ejecutaron catorce consultas Java sobre Hadoop 3: diez consultas de análisis, dos modelos de clasificación y dos modelos de regresión. Se logró conservar la salida y el archivo JAR de cada consulta. También se revisaron HDFS, YARN, JobHistory y el consumo de los contenedores. En clasificación, la regresión logística obtuvo mejores resultados generales que Gaussian Naive Bayes. En regresión, ridge tuvo una mejora pequeña frente al modelo lineal en la validación de 2025-I.

Palabras clave: Hadoop, MapReduce, ocupabilidad, clasificación, regresión.

Índice

1	Introducción	3
1.1	Contexto del problema	3
1.2	Objetivos del laboratorio	3
1.3	Organización del informe	3
2	Entorno Experimental y Despliegue de Hadoop	3
2.1	Arquitectura del entorno	3
2.2	Configuración de la máquina virtual de nodo único	3
2.3	Despliegue de Hadoop 3 mediante Docker Compose	3
2.4	Servicios de Hadoop y persistencia de datos	3
2.5	Configuración de HDFS y ejecución de MapReduce	3
2.6	Verificación del entorno	4
3	Descripción del Dataset	4
3.1	Fuente, publicación y actualización	4
3.2	Descripción general del dataset	4
3.3	Estructura, campos y tipos de datos	4
3.4	Preprocesamiento y carga en HDFS	4
3.5	Antecedentes e investigaciones relacionadas	4
3.5.1	Investigaciones previas sobre el dataset	4

3.5.2	Artículos que utilizaron el dataset	4
4	Metodología	4
4.1	Proceso de procesamiento distribuido	4
4.2	Diseño de preguntas y consultas	4
4.3	Flujo de ejecución	5
4.4	Métricas de evaluación	5
5	Consultas MapReduce	5
5.1	Consultas que emplean dos o más campos	5
5.1.1	Consulta 1: arribos nacionales y extranjeros	5
5.1.2	Consulta 2: TNOH por departamento y año	5
5.1.3	Consulta 3: TNOH y TNOC por clase	5
5.1.4	Consulta 4: empleo por capacidad	6
5.1.5	Consulta 5: estacionalidad mensual	6
5.2	Medidas estadísticas	6
5.2.1	Consulta 6: media, mediana y desviación estándar de TNOH	6
5.3	Búsqueda de subtexto	6
5.3.1	Consulta 7: búsqueda sobre campos de texto	6
5.4	Valores máximos y mínimos	7
5.4.1	Consulta 8: extremos de TNOH por año	7
5.5	Consultas con MapReduce encadenado	7
5.5.1	Consulta 9: cambio de ocupación entre 2019 y 2024	7
5.5.2	Consulta 10: participación de arribos extranjeros	7
6	Consultas mediante Modelos de Clasificación	7
6.1	Consulta 11: Gaussian Naive Bayes	7
6.2	Consulta 12: regresión logística	8
6.3	Comparación de modelos de clasificación	8
7	Consultas mediante Modelos de Regresión	8
7.1	Consulta 13: regresión lineal	8
7.2	Consulta 14: regresión ridge	8
7.3	Comparación de modelos de regresión	8
8	Monitoreo de Recursos	9
8.1	Recursos observados durante la ejecución	9
8.2	Consumo de CPU	9
8.3	Consumo de memoria	9
8.4	Tiempo de ejecución	10
8.5	Análisis del comportamiento de Hadoop	10
9	Resultados y Discusión	10
9.1	Resultados de las consultas MapReduce	10
9.2	Resultados de clasificación	10
9.3	Resultados de regresión	10
9.4	Discusión general	11
10	Conclusiones	11
A	Anexo: Código fuente Java y archivos JAR	11
B	Anexo: Inputs y outputs de las consultas	11
C	Anexo: Docker Compose y configuración de Hadoop	11
D	Anexo: Documentación adicional	11
	Bibliografía	11

1. Introducción

1.1. Contexto del problema

Los indicadores de hospedaje cambian por mes, departamento y tipo de establecimiento. Al juntar varios años aparecen diferencias de estructura, valores repetidos por nivel de agregación y un volumen que ya no resulta cómodo para revisar fila por fila. Por eso se usó Hadoop MapReduce. El objetivo no fue solo obtener números, sino dejar una ejecución repetible y revisar cada salida con datos reales.

1.2. Objetivos del laboratorio

Se buscó consolidar los archivos de 2019 a 2025-I, cargar una única entrada a HDFS y resolver catorce preguntas con Java MapReduce. También se compararon dos clasificadores y dos regresores usando una separación temporal. Finalmente, se registró el estado del clúster y el uso de recursos durante una ejecución.

1.3. Organización del informe

Primero se explica el entorno y la preparación del dataset. Después se muestra la metodología y las diez consultas de análisis. Luego se presentan los modelos de clasificación y regresión. Al final se revisan el monitoreo, los resultados principales, las limitaciones y las conclusiones.

2. Entorno Experimental y Despliegue de Hadoop

2.1. Arquitectura del entorno

Se usó un clúster Hadoop 3.5.0 en Docker Compose. El despliegue tuvo un NameNode, un DataNode, un ResourceManager, un NodeManager y un HistoryServer. Todos los servicios se ejecutaron en una sola máquina. Así se logró probar HDFS y YARN sin presentar el entorno como un clúster de producción.

2.2. Configuración de la máquina virtual de nodo único

El NodeManager informó 2 GB de memoria física para contenedores y 8 vcores. El nodo se mantuvo saludable. Como solo hubo un nodo, los mapas y reducers compartieron el mismo equipo. Esto comprueba el flujo, pero no permite medir escalamiento horizontal.

2.3. Despliegue de Hadoop 3 mediante Docker Compose

Los contenedores ya estaban definidos en `hadoop3/compose.yaml`. Se inició el entorno con el script del proyecto y se hizo una prueba rápida antes de ejecutar las consultas. No se reconstruyeron imágenes. Se usaron los servicios existentes y sus volúmenes persistentes.

2.4. Servicios de Hadoop y persistencia de datos

HDFS guardó la entrada y las salidas. YARN administró las aplicaciones MapReduce. JobHistory conservó el detalle de los trabajos terminados. El NameNode mostró 203 bloques replicados y un DataNode activo.

2.5. Configuración de HDFS y ejecución de MapReduce

Se usó `hadoop3/scripts/run-job.sh`. El script copia el código al NameNode, compila las clases Java, arma el JAR, sube el CSV a HDFS y ejecuta el Driver. Si la salida termina con `_SUCCESS`, exporta el resultado y el JAR al host. Para repetir todo se creó `labs/labca1/queries/run-all.sh`.

2.6. Verificación del entorno

Las catorce consultas terminaron con estado `SUCCESS`. Se observaron 33 aplicaciones finalizadas porque las consultas 9 y 10 usan dos etapas y también quedaron ejecuciones de prueba anteriores. `JobHistory` confirmó que la consulta 14 lanzó dos mapas y un reducer sin intentos fallidos.

3. Descripción del Dataset

3.1. Fuente, publicación y actualización

El dataset de Indicadores de Ocupabilidad del Ministerio de Comercio Exterior y Turismo (MINCETUR) se obtuvo de la Plataforma Nacional de Datos Abiertos [1].

3.2. Descripción general del dataset

El archivo consolidado cubre desde enero de 2019 hasta junio de 2025. Contiene 38 730 registros y 24 columnas. Se encontraron 25 departamentos en cada mes. El año 2025 se trató como primer semestre y no como un año completo.

3.3. Estructura, campos y tipos de datos

Los campos principales son año, mes, clase, categoría, departamento, cantidad de establecimientos, habitaciones, plazas-cama, TNOH, TNOC, arribos, pernoctaciones y empleo. Los archivos de 2019 a 2023 tenían 23 columnas. Los archivos recientes añadían `FECHA_CORTE`. La cabecera final usa nombres sin espacios y mantiene todos los indicadores en un orden fijo.

3.4. Preprocesamiento y carga en HDFS

Los archivos no tenían exactamente la misma estructura. Los datos de 2019 a 2023 usaban 23 columnas, mientras que los archivos más recientes añadían la fecha de corte. Por eso, se creó un archivo consolidado con 24 columnas. También se cambió la coma decimal por un punto y se guardó el resultado en UTF-8. Al final, se obtuvieron 38 730 registros entre enero de 2019 y junio de 2025.

Los archivos originales se leyeron como Latin-1 y se conservaron intactos. El script validó la cabecera, la cantidad de columnas, el rango de años y los campos numéricos. Para agregaciones generales se usaron solo las filas totales de clase y categoría, identificadas como `TT/TT`. Ese filtro fue necesario porque el dataset repite totales en distintos niveles.

3.5. Antecedentes e investigaciones relacionadas

3.5.1. Investigaciones previas sobre el dataset

3.5.2. Artículos que utilizaron el dataset

4. Metodología

4.1. Proceso de procesamiento distribuido

Cada consulta lee el CSV canónico mediante un parser común. El Mapper descarta la cabecera y las filas que no cumplen el filtro. Después emite una clave y los valores necesarios. El Reducer suma, promedia, busca extremos o entrena el modelo. Las consultas 9 y 10 encadenan dos jobs. Las demás usan un job.

4.2. Diseño de preguntas y consultas

Las preguntas cubren evolución temporal, diferencias territoriales, clases de hospedaje, capacidad, estacionalidad, estadística descriptiva, búsqueda de texto, extremos, cambios entre periodos y participación extranjera. Las cuatro últimas consultas usan la TNOH como etiqueta o variable objetivo.

4.3. Flujo de ejecución

El orquestador ejecuta las consultas en orden y se detiene ante el primer error. Cada salida quedó en su carpeta de resultados, junto al JAR. También se creó un resumen TSV con clase principal, estado, ruta HDFS y artefactos locales.

4.4. Métricas de evaluación

Para clasificación se usaron accuracy, precisión, recall, F1 y log loss. Para regresión se usaron MAE, RMSE y R^2 . El entrenamiento tomó 1 500 filas TT/TT de 2019 a 2023. La prueba usó 300 filas de 2024 y la validación reciente usó 150 filas de 2025-I. La mediana de TNOH del entrenamiento fue 21.71 y definió las clases alta y baja. La estandarización se calculó solo con entrenamiento.

5. Consultas MapReduce

En cada consulta se indican el tipo, la pregunta, los campos, la solución, los archivos de código, la entrada, una salida real, una interpretación y una limitación. La entrada fue el CSV consolidado en todos los casos.

5.1. Consultas que emplean dos o más campos

5.1.1. Consulta 1: arribos nacionales y extranjeros

Tipo y pregunta. Agregación por año. ¿Cómo evolucionaron los arribos nacionales y extranjeros? Se usaron ANIO, TOTAL_ARRIBOS_NAC y TOTAL_ARRIBOS_EXT.

Solución y código. El Mapper filtró TT/TT y emitió el año con ambos totales. El Reducer sumó cada origen. Se usaron las clases `ArrivalsByOriginMapper`, `Reducer` y `Driver`.

Input y output.

```
2019 national=54897127 foreign=8264767
2020 national=34588588 foreign=1778619
2024 national=50875669 foreign=6690972
2025 national=26355966 foreign=3610473
```

Interpretación y limitación. Se observó una caída fuerte en 2020 y una recuperación posterior. En 2024 los arribos nacionales quedaron cerca de 2019, pero los extranjeros siguieron por debajo. El valor de 2025 solo incluye seis meses y no se puede comparar directamente con un año completo.

5.1.2. Consulta 2: TNOH por departamento y año

Tipo y pregunta. Promedio agrupado. ¿Cuál fue la TNOH media de cada departamento por año? Se usaron año, departamento y TNOH.

Solución y código. El Mapper emitió la clave año-departamento y el Reducer calculó suma entre cantidad. Se usaron las clases `OccupancyByDepartmentMapper`, `Reducer` y `Driver`.

Input y output.

```
2019;CALLAO mean_tnoh=40.9142
2020;CALLAO mean_tnoh=25.7092
2024;CALLAO mean_tnoh=41.5950
2024;LIMA mean_tnoh=23.5750
2024;LORETO mean_tnoh=12.9917
```

Interpretación y limitación. Callao llegó a 41.5950 en 2024, mientras Loreto obtuvo 12.9917. El promedio resume meses con tamaños de oferta distintos y no equivale a una media ponderada por habitaciones.

5.1.3. Consulta 3: TNOH y TNOC por clase

Tipo y pregunta. Comparación por grupos. ¿Cómo cambian la TNOH y la TNOC según la clase? Se usaron año, clase y ambos porcentajes. Se tomó la categoría total y se excluyó la clase total.

Solución y código. El Mapper agrupó año-clase y el Reducer calculó las dos medias. Se usaron las clases `OccupancyByClass`.

```
2019;HOTEL mean_tnoh=31.2412 mean_tnoc=26.3323
2020;HOTEL mean_tnoh=21.1950 mean_tnoc=16.1440
2024;HOTEL mean_tnoh=32.4184 mean_tnoc=27.6639
2024;RESORT mean_tnoh=46.7742 mean_tnoc=48.2812
```

Interpretación y limitación. Resort mantuvo medias altas, mientras Hotel cayó en 2020 y luego se recuperó. Las clases no tienen la misma cantidad de establecimientos, así que la salida no compara tamaños totales.

5.1.4. Consulta 4: empleo por capacidad

Tipo y pregunta. Razón entre campos. ¿Cuántos empleos existen por cada cien habitaciones en cada departamento y año? Se usaron año, departamento, empleo y habitaciones.

Solución y código. El Mapper emitió empleo y habitaciones TT/TT. El Reducer sumó ambos campos. Después calculó la razón por habitación y la multiplicó por 100. Se usaron las clases `EmploymentCapacity`.

```
2019;LIMA employment_per_100_rooms=74.3232
2024;LIMA employment_per_100_rooms=54.3562
2024;CALLAO employment_per_100_rooms=28.4118
2024;TACNA employment_per_100_rooms=15.0163
```

Interpretación y limitación. Lima presentó más empleos por cada cien habitaciones que Callao y Tacna. La razón no mide calidad del empleo ni horas trabajadas; solo relaciona los totales informados.

5.1.5. Consulta 5: estacionalidad mensual

Tipo y pregunta. Suma por mes. ¿Qué meses concentran más arribos y pernoctaciones? Se usaron mes, arribos y pernoctaciones TT/TT.

Solución y código. El Mapper agrupó por mes y el Reducer sumó ambos indicadores. Se usaron las clases `Seasonality`.

```
01 arrivals=33585654 overnight_stays=43787718
02 arrivals=32795352 overnight_stays=42942772
07 arrivals=26760124 overnight_stays=35417748
09 arrivals=26098980 overnight_stays=33472076
```

Interpretación y limitación. Enero y febrero tuvieron los mayores acumulados, mientras septiembre tuvo el menor valor. Enero a junio incluyen siete años, pero julio a diciembre solo seis. La suma no está balanceada.

5.2. Medidas estadísticas

5.2.1. Consulta 6: media, mediana y desviación estándar de TNOH

Tipo y pregunta. Estadística descriptiva. ¿Cuál es la cantidad, media, mediana y desviación estándar poblacional de TNOH?

Solución y código. El Mapper emitió cada TNOH TT/TT bajo una clave común. El Reducer ordenó 1 950 valores. Se usaron las clases `OccupancyStatistics`.

```
TNOH count=1950 mean=21.7433 median=21.3400 stddev=7.0678612717
```

Interpretación y limitación. La media y mediana quedaron cerca, y la desviación fue 7.0679 puntos. Un reducer pudo reunir los valores porque el dataset es pequeño. Para más datos convendría un método distribuido de cuantiles.

5.3. Búsqueda de subtexto

5.3.1. Consulta 7: búsqueda sobre campos de texto

Tipo y pregunta. Filtro por subtexto. ¿Qué registros contienen la palabra hotel en clase, categoría o departamento? La búsqueda no distingue mayúsculas, pero conserva los acentos.

Solución y código. `SubstringSearchMapper.java` recibió el texto como argumento y revisó los tres campos. No necesitó reducir. Devolvió el desplazamiento y la fila canónica completa.

```
2424268 ;2022;06;01;HOTEL;01;1 ESTRELLA;AMAZONAS;010000;1;20;42;...
2424367 ;2022;06;01;HOTEL;01;1 ESTRELLA;PIURA;200000;2;40;77;...
```

Interpretación y limitación. Se obtuvieron 11 525 coincidencias porque HOTEL aparece como clase en muchos registros. La búsqueda es literal; no aplica sinónimos, ranking ni normalización de acentos.

5.4. Valores máximos y mínimos

5.4.1. Consulta 8: extremos de TNOH por año

Tipo y pregunta. ¿Qué departamentos tuvieron la menor y mayor TNOH media de cada año? Se usaron año, departamento y TNOH.

Solución y código. El Mapper envió las filas TT/TT por año. El Reducer calculó la media departamental y eligió ambos extremos. Se usaron las clases `OccupancyExtremes`.

```
2019 min=TACNA:20.7342 max=CALLAO:40.9142
2020 min=TACNA:9.6425 max=CALLAO:25.7092
2024 min=LORETO:12.9917 max=CALLAO:41.5950
2025 min=AYACUCHO:14.1217 max=CALLAO:43.9917
```

Interpretación y limitación. Callao fue el máximo en seis años; en 2021 Piura quedó arriba. Los extremos usan promedios simples y 2025 solo cubre el primer semestre.

5.5. Consultas con MapReduce encadenado

5.5.1. Consulta 9: cambio de ocupación entre 2019 y 2024

Tipo y pregunta. Dos jobs. ¿Cuántos puntos porcentuales cambió la TNOH media de cada departamento entre 2019 y 2024?

Solución y código. El primer job calculó la media por año y departamento. El segundo unió 2019 con 2024 y restó. El Driver `OccupancyChangeDriver.java` coordinó ambas etapas.

```
LA LIBERTAD 2019=30.7350 2024=14.7108 change_pp=-16.0242
CALLAO 2019=40.9142 2024=41.5950 change_pp=0.6808
CUSCO 2019=26.9108 2024=28.2725 change_pp=1.3617
UCAYALI 2019=25.1383 2024=30.1825 change_pp=5.0442
```

Interpretación y limitación. La mayor caída fue Apurímac con $-18,7325$ puntos y la mayor subida fue Ucayali con $5,0442$. La diferencia describe una asociación temporal y no prueba la causa.

5.5.2. Consulta 10: participación de arribos extranjeros

Tipo y pregunta. Dos jobs. ¿Qué porcentaje de los arribos fue extranjero en cada departamento y periodo?

Solución y código. El primer job calculó el porcentaje mensual. El segundo promedió cuatro periodos. `ForeignShareDriver.java` coordinó las etapas.

```
CUSCO;PRE_2019 mean_foreign_share=63.3747
CUSCO;IMPACT_2020_2021 mean_foreign_share=20.5234
CUSCO;RECOVERY_2022_2024 mean_foreign_share=58.3507
CUSCO;CURRENT_2025_H1 mean_foreign_share=67.4504
```

Interpretación y limitación. En Cusco se observó una caída fuerte en 2020–2021 y recuperación posterior. Se promediaron porcentajes mensuales sin ponderar por arribos; no equivale a dividir los acumulados del periodo.

6. Consultas mediante Modelos de Clasificación

6.1. Consulta 11: Gaussian Naive Bayes

Pregunta, campos y código. Se clasificó la TNOH como alta o baja. Se usaron mes, establecimientos, habitaciones, camas, arribos por habitación, pernотaciones por cama y empleo por establecimiento. El Mapper preparó filas TT/TT y el Reducer entrenó con 2019–2023. Se usaron las clases `NaiveBayesClassification`.

```
TEST_2024 samples=300 accuracy=0.703333 precision=0.945946 recall=0.286885 f1=0.440252 log_loss=0.947039
VALIDATION_2025_H1 samples=150 accuracy=0.726667 precision=1.000000 recall=0.305085 f1=0.467532 log_loss=0.854541
```

Interpretación y limitación. Se obtuvo precisión alta, pero recall bajo. El modelo dejó sin detectar muchas observaciones altas. La independencia condicional es una limitación para indicadores relacionados.

6.2. Consulta 12: regresión logística

Pregunta, campos y código. Se respondió la misma pregunta con las mismas siete variables, estandarizadas solo con entrenamiento. El Reducer entrenó 200 iteraciones con tasa 0.1. Se usaron las clases `LogisticClassification`.

```
TEST_2024 samples=300 accuracy=0.840000 precision=0.836364 recall=0.754098 f1=0.793103 log_loss=0.498765
VALIDATION_2025_H1 samples=150 accuracy=0.800000 precision=0.871795 recall=0.576271 f1=0.693878 log_loss=0.474774
```

Interpretación y limitación. Se obtuvo un equilibrio más claro entre precisión y recall. El entrenamiento se centralizó en un reducer porque solo hubo 1 500 filas; esto no representa entrenamiento distribuido a gran escala.

6.3. Comparación de modelos de clasificación

Cuadro 1: Resultados de clasificación con salidas reales

Periodo	Modelo	Accuracy	Precisión	Recall	F1	Log loss
2024	Naive Bayes	0.7033	0.9459	0.2869	0.4403	0.9470
2024	Logística	0.8400	0.8364	0.7541	0.7931	0.4988
2025-I	Naive Bayes	0.7267	1.0000	0.3051	0.4675	0.8545
2025-I	Logística	0.8000	0.8718	0.5763	0.6939	0.4748

La regresión logística tuvo mayor accuracy, recall y F1, y menor log loss. Naive Bayes tuvo mayor precisión, pero perdió muchos positivos. Con estos datos, la regresión logística dejó el mejor resultado general.

7. Consultas mediante Modelos de Regresión

7.1. Consulta 13: regresión lineal

Pregunta, campos y código. Se estimó la TNOH con las mismas siete características y la misma separación temporal. El Reducer ajustó una regresión sin regularización. Se usaron las clases `LinearRegression`.

```
TRAINING samples=1500
TEST_2024 samples=300 mae=3.192221 rmse=4.092002 r_squared=0.643537
VALIDATION_2025_H1 samples=150 mae=3.020832 rmse=3.927314 r_squared=0.662158
```

Interpretación y limitación. Se obtuvo un MAE cercano a tres puntos. El R^2 fue 0.6435 en 2024 y 0.6622 en 2025-I. El modelo solo representa relaciones lineales.

7.2. Consulta 14: regresión ridge

Pregunta, campos y código. Se estimó la misma variable con regularización L2, $\lambda = 1$, sin regularizar el intercepto. Se usaron las clases `RidgeRegression`.

```
TRAINING samples=1500
TEST_2024 samples=300 mae=3.194316 rmse=4.072566 r_squared=0.646915
VALIDATION_2025_H1 samples=150 mae=3.013257 rmse=3.879584 r_squared=0.670320
```

Interpretación y limitación. Ridge redujo un poco el RMSE y mejoró el R^2 . Solo se evaluó un valor de λ , así que no se puede afirmar que sea la mejor regularización posible.

7.3. Comparación de modelos de regresión

Cuadro 2: Resultados de regresión con salidas reales

Periodo	Modelo	MAE	RMSE	R^2
2024	Lineal	3.1922	4.0920	0.6435
2024	Ridge	3.1943	4.0726	0.6469
2025-I	Lineal	3.0208	3.9273	0.6622
2025-I	Ridge	3.0133	3.8796	0.6703

La diferencia fue pequeña. Ridge quedó mejor en RMSE y R^2 . En MAE, el lineal fue apenas mejor en 2024 y ridge fue mejor en 2025-I.

8. Monitoreo de Recursos

8.1. Recursos observados durante la ejecución

Las interfaces mostraron un NameNode activo, un NodeManager saludable y 33 aplicaciones completadas. La Figura 1 reúne HDFS y YARN. La Figura 2 muestra un job real de la consulta 14.

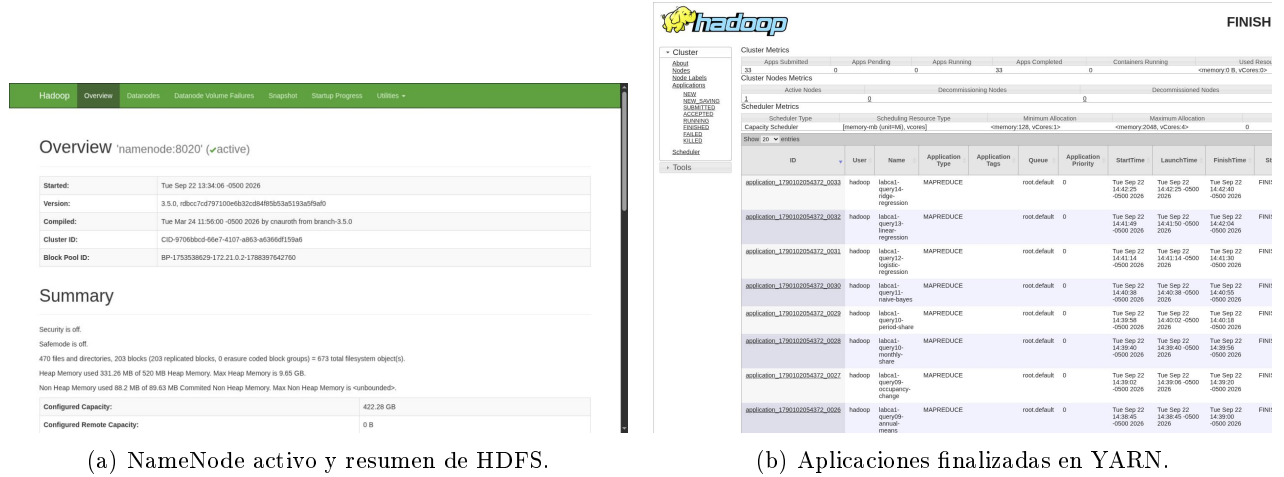


Figura 1: Estado de los servicios principales de Hadoop.



Figura 2: Recursos del nodo y detalle de una ejecución.

8.2. Consumo de CPU

La medición se tomó el 22 de septiembre de 2026 a las 14:41:56, mientras se ejecutaba la consulta 13. El NodeManager registró 1103.66 % de CPU. Docker mide 100 % por núcleo, así que el valor indica uso de varios núcleos.

8.3. Consumo de memoria

El NodeManager usó 1.193 GiB. El NameNode usó 992.1 MiB, el ResourceManager 475.5 MiB, el History-Server 346.9 MiB y el DataNode 335.5 MiB. Es una medición puntual, no un promedio.

Uso de recursos durante la ejecución Hadoop							
Consulta activa: query13 – regresión lineal							
2026-09-22T14:41:56-05:00							
NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS	
hadoop3-lab-nodemanager-1	1103.66%	1.1936iB / 38.586iB	3.09%	94.3MB / 36MB	5.99MB / 27.8MB	407	
hadoop3-lab-historyserver-1	0.14%	346.9MiB / 38.586iB	0.88%	185kB / 113kB	979kB / 1.26MB	79	
hadoop3-lab-resourcemanager-1	2.00%	475.5MiB / 38.586iB	1.26%	2.75MB / 1.42MB	25.2MB / 24.2MB	301	
hadoop3-lab-datanode-1	0.25%	335.5MiB / 38.586iB	0.85%	116MB / 97.8MB	2.39MB / 110MB	82	
hadoop3-lab-namenode-1	1.46%	992.1MiB / 38.586iB	2.51%	7.95MB / 85.4MB	147MB / 46.6MB	124	
en_mancha_redis	0.63%	17.36MiB / 38.586iB	0.04%	176kB / 126B	8.74MB / 0B	6	
en_mancha_postgres	0.00%	40.42MiB / 38.586iB	0.10%	176kB / 126B	24.5MB / 102kB	6	

Figura 3: Uso de recursos durante la consulta 13.

8.4. Tiempo de ejecución

JobHistory registró 10 segundos para el job ridge de la consulta 14: empezó a las 19:42:30 UTC y terminó a las 19:42:40. No incluye copia del código, compilación ni exportación.

8.5. Análisis del comportamiento de Hadoop

Los trabajos pequeños tienen un costo fijo al iniciar mapas, shuffle y reducer. El clúster procesó el CSV sin fallos y dejó trazas. El dataset y el único NodeManager prueban el flujo completo, pero no demuestran escalabilidad horizontal.

9. Resultados y Discusión

9.1. Resultados de las consultas MapReduce

Las salidas mostraron la caída de 2020, diferencias entre departamentos y una recuperación desigual. Callao presentó TNOH alta en varios años. En el cambio 2019–2024, Apurímac tuvo la mayor caída y Ucayali la mayor subida. Cusco mostró una caída de participación extranjera en 2020–2021 y recuperación posterior.

Las cifras se contrastaron con Python en modo de solo lectura. Los totales de las consultas 1 y 5 y las estadísticas de la consulta 6 coincidieron. También se confirmaron 1 500 muestras de entrenamiento, 300 de prueba y 150 de validación. Sumar todos los niveles cuadruplicaba aproximadamente algunos totales; el filtro TT/TT evitó ese doble conteo.

9.2. Resultados de clasificación

La regresión logística fue más equilibrada. En 2024 obtuvo F1 de 0.7931 frente a 0.4403 de Naive Bayes. En 2025-I obtuvo 0.6939 frente a 0.4675. Naive Bayes tuvo precisión alta, pero recall bajo.

9.3. Resultados de regresión

Ridge mejoró el RMSE de 4.0920 a 4.0726 en 2024 y de 3.9273 a 3.8796 en 2025-I. También elevó un poco el R^2 . La mejora fue pequeña.

9.4. Discusión general

Se logró responder las catorce consultas con una entrada y una ejecución repetible. Las consultas encadenadas mostraron cómo una salida intermedia alimenta otro job. En los modelos se usó una división temporal para entrenar con el pasado y evaluar periodos posteriores.

Las limitaciones principales son el dataset pequeño, un solo nodo, algunos promedios no ponderados y el semestre incompleto de 2025. Además, los modelos se entrenaron en un reducer. Fue suficiente para 1 950 filas TT/TT, pero no sería adecuado para un modelo muy grande.

10. Conclusiones

Se obtuvo un CSV canónico de 38 730 registros sin modificar los siete archivos originales. El filtro TT/TT evitó doble conteo. Las catorce consultas terminaron correctamente y conservaron resultado y JAR.

Se observó una caída de varios indicadores en 2020 y una recuperación diferente entre departamentos. Las consultas encadenadas compararon 2019 con 2024 y agruparon la participación extranjera por periodos.

La regresión logística obtuvo el mejor equilibrio en clasificación. Ridge mejoró ligeramente RMSE y R^2 . Estos resultados describen este dataset y no prueban causas ni garantizan el mismo desempeño en otros periodos.

Finalmente, se logró comprobar HDFS, YARN y JobHistory en un nodo. Las capturas muestran trabajos y consumo reales. El entorno sirve para el laboratorio, pero no demuestra escalabilidad horizontal.

A. Anexo: Código fuente Java y archivos JAR

El código está en `queries/common/src` y `queries/query01` a `queries/query14`. Cada consulta conserva su JAR en `resultados/queryXX/queryXX.jar`.

B. Anexo: Inputs y outputs de las consultas

La entrada es `data/Indicadores_ocupabilidad_2019_2025.csv`. Cada salida completa está en `queries/queryXX/resultados/queryXX/resultado.txt`. `queries/execution-summary.tsv` registra las catorce ejecuciones.

C. Anexo: Docker Compose y configuración de Hadoop

La definición está en `hadoop3/compose.yaml`. Los scripts están en `hadoop3/scripts/`.

D. Anexo: Documentación adicional

Cada consulta incluye un README. Las evidencias se documentan en `labs/labca1/images/README.md`.

Referencias

- [1] Ministerio de Comercio Exterior y Turismo (MINCETUR). Indicadores de ocupabilidad. Plataforma Nacional de Datos Abiertos. Consultado el 22 de septiembre de 2026.